

ProjectFlow: A Project Management App

CSE299-Group-3

Safat Uddin (2311966642), Farhan Karim (2311970642), Faizur Rahman Zunayed (2312137642)

Department of Computer Science and Engineering

Final Project Report

Abstract—This report presents the design, implementation, and features of ProjectFlow, a comprehensive project management system built using modern web technologies. The system facilitates team collaboration, streamlines project workflows, and provides real-time visibility into task progress. The application follows a three-tier architecture with React frontend, Express.js backend, and MongoDB database. Key features include workspace management, kanban boards, comprehensive task management with subtasks and attachments, real-time notifications, and role-based access control. The system demonstrates proficiency in full-stack development and provides a production-ready solution for team project management.

Index Terms—Project Management, React, Express.js, MongoDB, Real-time Collaboration, Task Management

I. INTRODUCTION

Modern project management requires efficient collaboration tools that provide real-time visibility into project progress. **ProjectFlow** addresses this need by offering a comprehensive platform for organizing work hierarchically (Workspace → Project → Task), enabling team communication, and streamlining workflows through intuitive interfaces and real-time updates.

A. Project Purpose

The primary purpose of ProjectFlow is to facilitate team collaboration, streamline project workflows, and provide real-time visibility into task progress. The system enables teams to organize work into workspaces, manage projects with kanban boards, and track individual task details with comprehensive collaboration features.

B. Target Users

- **Team Leaders:** Manage workspaces, oversee projects, assign tasks
- **Project Managers:** Create projects, track progress, manage team members
- **Team Members:** Complete assigned tasks, collaborate via comments, upload files
- **Stakeholders:** View project progress, monitor team activity

C. Core Objectives

- 1) **Organization:** Structure work hierarchically (Workspace → Project → Task)
- 2) **Collaboration:** Enable team communication through comments, mentions, attachments
- 3) **Visibility:** Real-time activity tracking and notifications
- 4) **Efficiency:** Streamline workflows with drag-and-drop, inline editing, quick actions

- 5) **Access Control:** Role-based permissions and task-level access restrictions

II. TECHNICAL ARCHITECTURE

A. Technology Stack

TABLE I: Technology Stack

Layer	Technologies
Backend	Node.js, Express.js 4.21.2, MongoDB, Mongoose 8.19.1
Authentication	JWT, Passport.js, Google OAuth 2.0, bcrypt
Frontend	React 19.1.1, React Router 7.9.2, TanStack Query 5.90.2
UI Components	Radix UI, shadcn/ui, Tailwind CSS 4.1.13
Real-time	DnD Kit 6.3.1, Recharts 2.15.4
Development Tools	Vite, Zod, React Hook Form 7.64.0

B. Architecture Pattern

The system follows a three-tier architecture pattern:

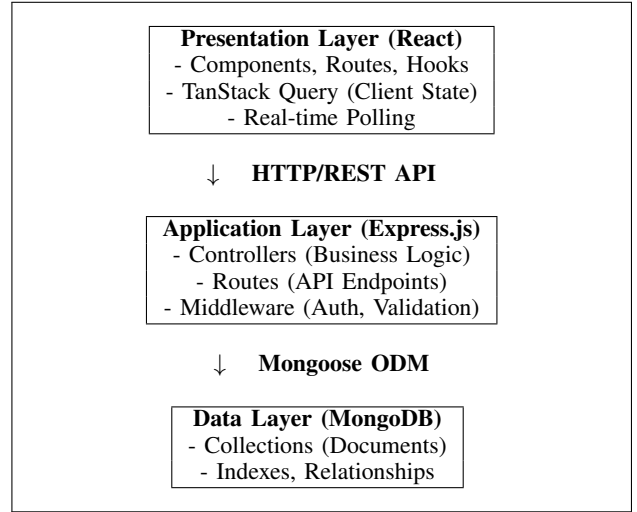


Fig. 1: Three-tier architecture diagram

C. System Components

1) Backend Components (6 Controllers):

- **Auth Controller:** User authentication, OAuth, email verification
- **User Controller:** Profile management, search, settings
- **Workspace Controller:** Workspace CRUD, member management, invitations
- **Project Controller:** Project CRUD, member assignment, archiving

- Task Controller: Task operations, subtasks, comments, attachments
- Notification Controller: Notification management, read status

2) Frontend Components:

- **Pages (15+):** Route-based pages for different views
- **UI Components (20+):** Reusable UI components
- **Feature Components (15+):** Task, project, workspace specific
- **Layout Components:** Header, sidebar, layouts
- **Hooks (6):** Custom hooks for API operations

III. SYSTEM FEATURES

A. Authentication & Authorization

The authentication system provides multiple login options and robust security measures:

- Email/password registration and login
- Google OAuth 2.0 integration
- Email verification system
- Password reset via email
- JWT token-based authentication (24h expiration)
- Role-based access control (Owner, Admin, Member)

Security Measures:

- bcrypt password hashing (10 rounds)
- JWT secret key protection
- Arcjet rate limiting
- CORS configuration
- Secure password reset tokens

B. Workspace Management

TABLE II: Workspace Permission Levels

Role	Edit Workspace	Manage Members	Create Projects	Delete Workspace
Owner	✓	✓	✓	✓
Admin	✗	✓	✓	✗
Member	✗	✗	✗	✗

C. Project Management

Projects are organized hierarchically under workspaces with the following features:

- Create projects within workspaces
- Set project status (Not Started, In Progress, Completed, On Hold)
- Track project progress (0-100%)
- Set start and end dates
- Add/remove project members
- Archive/restore projects
- Kanban board view with drag-and-drop

D. Task Management

The task management system provides comprehensive features for individual task handling:

1) Core Task Features:

- Create tasks within projects with editable title
- Rich text description support
- Assign multiple users to tasks
- Priority levels (Low, Medium, High, Critical)
- Due dates with calendar picker
- Task status (To Do, In Progress, Done)

2) Access Control:

- Only assigned members can view task details
- Non-assigned members receive notification when attempting access
- Prevents unauthorized access at both task and project levels

3) Subtasks System:

- Add unlimited subtasks
- Mark subtasks complete/incomplete
- Edit subtask titles
- Delete subtasks
- Progress indicator (X/Y completed)

4) Comments System:

- Add comments to tasks with user mentions (@username)
- Edit and delete own comments
- Timestamp display with user avatars
- Real-time comment updates (1 second polling)

E. Notification System

TABLE III: Notification Types

Type	Description
Task Assigned	User assigned to a task
Task Updated	Task details modified
Comment Added	New comment on watched task
File Uploaded	File attachment added
Subtask Added	New subtask created
User Mentioned	@username used in comment
Project Updates	Project status changes

IV. IMPLEMENTATION DETAILS

A. Backend Architecture

The backend follows a modular structure with clear separation of concerns:

```
backend/
|-- controllers/ # Business logic (6 files)
|-- models/     # Database schemas (9 models)
|-- routes/     # API endpoints (7 routers)
|-- middleware/ # Custom middleware (2 files)
|-- libs/       # Utilities/Helper Tools (5 files)
|-- uploads/    # File storage (2 folders)
```

Fig. 2: Backend directory structure

Request Flow:

- 1) Client Request
- 2) Express Router
- 3) Middleware (CORS, JSON, Compression, Morgan)
- 4) Auth Middleware (JWT Verification)
- 5) Validation (Zod Schema)

- 6) Controller (Business Logic)
- 7) Model (Mongoose Query)
- 8) Database (MongoDB)
- 9) Response (JSON)

B. Frontend Architecture

The frontend uses a component-based architecture with React:

```
frontend/app/
|-- routes/           # Pages(file-based routing)
|-- components/       # Reusable components
|   |-- ui/           # Base UI (shadcn)
|   |-- task/         # Task-specific
|   |-- project/      # Project-specific
|   |-- workspace/    # Workspace-specific
|-- hooks/            # Custom React hooks
|-- lib/              # Utilities
|-- provider/         # Context providers
|-- types/            # TypeScript definitions
```

Fig. 3: Frontend directory structure

V. DATABASE SCHEMA

A. Collections Overview

The system uses 9 MongoDB collections:

- **users:** User account information
- **workspaces:** Workspace data and memberships
- **workspace_invites:** Pending workspace invitations
- **projects:** Project definitions and metadata
- **tasks:** Task information and relationships
- **comments:** Task comments and discussions
- **activities:** System activity logging
- **notifications:** User notifications
- **verifications:** Email verification tokens

B. Key Schema Definitions

TABLE IV: Task Schema Fields

Field	Type	Description
_id	ObjectId	Unique task identifier
title	String	Task title
description	String	Detailed task description
project	ObjectId	Reference to parent project
assignees	[ObjectId]	Array of assigned users
status	String	To Do / In Progress / Done
priority	String	Low / Medium / High / Critical
subtasks	Array	Subtask objects with completion status
attachments	Array	File and link attachments
createdBy	ObjectId	Task creator
timestamps	Boolean	createdAt, updatedAt

C. Database Relationships

- **Workspace → Projects (1:N):** One workspace has many projects
- **Project → Tasks (1:N):** One project has many tasks
- **Task → Comments (1:N):** One task has many comments
- **User → Tasks (N:M):** Users can be assigned to multiple tasks

- **User → Workspaces (N:M):** Users can belong to multiple workspaces

VI. API DOCUMENTATION

A. API Endpoints Summary

The system provides over 70 API endpoints across six main categories:

TABLE V: API Endpoint Categories

Category	Endpoints	Example
Authentication	7	POST /auth/signup, POST /auth/login
Users	6	GET /users/me, PUT /users/userId
Workspaces	11	POST /workspaces, GET /workspaces
Projects	9	POST /projects, GET /projects/:projectId
Tasks	25+	POST /tasks, GET /tasks/:taskId
Notifications	4	GET /notifications, PUT /notifications/read-all

B. Authentication Mechanism

All protected endpoints require JWT token in the Authorization header:

Authorization: Bearer <token>

Tokens are obtained through:

- POST /auth/login (email/password)
- POST /auth/signup (registration)
- GET /auth/google/callback (OAuth)

VII. SECURITY & AUTHENTICATION

A. Authentication Flows

Local Authentication Flow:

- 1) User submits signup form
- 2) Backend hashes password with bcrypt (10 rounds)
- 3) Creates user in database
- 4) Sends verification email via SendGrid
- 5) User clicks verification link
- 6) Generates JWT token (24-hour expiration)
- 7) Returns token to client for storage

Google OAuth Flow:

- 1) User clicks "Sign in with Google"
- 2) Redirect to Google authentication
- 3) User authorizes application
- 4) Google callback to backend
- 5) Create/find user in database
- 6) Generate JWT token
- 7) Redirect to dashboard with token

B. Security Measures

- **Password Security:** bcrypt hashing, minimum 8 characters
- **Token Security:** JWT with secret key, 24-hour expiration
- **API Security:** Arcjet rate limiting, CORS whitelist, input validation
- **Access Control:** Role-based permissions, task-level access control
- **Data Protection:** Environment variables, secure file uploads

VIII. CONCLUSION

A. Project Achievements

ProjectFlow successfully delivers a comprehensive project management solution with:

- **Complete Feature Set:** Full task lifecycle management with collaboration tools
- **Modern Technology Stack:** React 19, Express.js, MongoDB with best practices
- **User-Centric Design:** Intuitive interface with real-time feedback
- **Robust Security:** JWT authentication, OAuth integration, role-based permissions
- **Scalable Architecture:** Modular design with efficient database schema

B. Technical Insights

Key lessons learned during development:

- Real-time polling provides effective updates for moderate-scale applications
- TanStack Query simplifies complex state management
- Mongoose .populate() requires careful handling for performance
- Comprehensive activity tracking provides valuable user insights
- Multi-level access control enhances application security

C. Final Assessment

ProjectFlow represents a fully-functional, production-ready project management application built with modern web technologies. The system demonstrates proficiency in:

- **Backend Development:** RESTful API design, database modeling, authentication
- **Frontend Development:** React component architecture, state management
- **System Design:** Real-time features, access control, scalability
- **DevOps Practices:** Deployment strategy, security implementation

The application is ready for real-world deployment and can scale to support growing teams and increasing workloads.